

LOADLENS

Во что вам обходится каждый запрос к вашему API

Инфраструктура считает запросы штуками. Но один и тот же адрес стоит по-разному в десятки раз — и за этот разброс вы платите серверами.

Синявский Станислав · МГТУ им. Н. Э. Баумана · winlast.github.io/loadlens

ssd-tula@mail.ru

Быстрая прямая связь в Telegram: @winlast

Пользователь ждёт из-за чужого запроса

Человек нажимает «посмотреть баланс». Обычно это доли секунды, иногда — три. Он ничего не сделал не так: в этот момент кто-то другой заказал выписку за год, и его тяжёлый запрос занял сервер целиком.

Компании лечат это деньгами — покупают кратно больше мощностей, чтобы редкие тяжёлые запросы не ломали отклик остальным. Это работает и стоит дорого.

Почему так выходит

- Балансировщик распределяет запросы, не зная их стоимости
- Для него пять лёгких обращений и пять тяжёлых неразличимы
- А разница между ними — десятки раз

Это не чья-то ошибка, а устройство почти всякого современного сервиса.

Стоимость запроса не определяется его адресом

Это чужая база данных — PostgreSQL за PostgREST. Нашего кода в ней нет ни строки. Все четыре обращения идут по одному и тому же адресу:

запрос — адрес один и тот же	время
/orders?id=eq.500	1,7 мс
/orders?limit=5	2,0 мс
/orders?limit=20000	49,1 мс
/orders?order=amount.desc&limit=20000	123,1 мс

в 72 раза

разница между самым дешёвым и самым дорогим обращением по одному адресу

Настройками это не лечится: вес задаётся на адрес, а различие лежит внутри адреса. Настраивать нечего.

Почти у половины сервисов в настоящем облаке

Microsoft опубликовала журнал своей облачной платформы. Это настоящие клиенты, а не наш стенд — мы к этим данным не прикасались.

44,6 %

обработчиков расходятся по времени не меньше чем в 10 раз

11,7 %

расходятся не меньше чем в 100 раз

в 8 раз

типичный разброс — и мы говорим это прямо, а не берём рекордные примеры

Объём данных: 32 968 обработчиков, 1 801 682 426 вызовов.

Проблема существует не только у нас. Значит, есть кому продавать.

Узнать стоимость заранее — и разложить запросы

Мы вычисляем стоимость каждого запроса до того, как он попадёт на сервер, — из обычного журнала обращений, без единой правки в коде клиента.

1 Показать

Читает журнал, который сервер уже пишет. Ничего не устанавливается, код не меняется, доступ к системе не нужен. За минуту показывает, есть ли проблема и сколько она стоит.

Готово. Бесплатно. Работает сегодня.

2 Починить

Раскладывает обращения так, чтобы тяжёлые не задерживали лёгкие. Ставится внутрь того сервера, который у клиента уже стоит, либо отдельной программой.

Стабильно работает и замерено. Для боевой эксплуатации — нужны реальные клиенты и немного времени

Порядок именно такой: никто не поставит незнакомую программу в путь боевых запросов. Вход — диагностика с нулевым риском.

ЧТО ЭТО ДАЁТ

Меньше серверов при том же качестве отклика

656 640 ₽

в год для сервиса на 1000 запросов в секунду
со строгим требованием к отклику

27 серверов
было нужно

17 серверов
стало достаточно

Это 37 % расходов на серверы. Расчёт по публичному прейскуранту Yandex Cloud — проверяется в один клик.

Второй эффект — пользовательский: в нашем замере каждый третий быстрый запрос тормозил дольше двух десятых секунды. Стало — ни одного.

Под нагрузкой задержка у нас не растёт



Как это читать

Чем больше людей одновременно пользуются сервисом, тем дольше ждут те, кому не повезло.

У обычной балансировки ожидание вырастает почти в двенадцать раз.

У нас остаётся на месте: тяжёлые обращения просто не попадают туда, где мешают.

Именно отсюда берётся экономия на серверах: чтобы уложиться в обещание по скорости, обычной схеме нужно больше машин.

Проверить можно прямо сейчас, за минуту

Не надо верить нам на слово и не надо ничего устанавливать. Инструмент открывается в браузере как обычная страница.

winlast.github.io/loadlens

- Перетащить журнал обращений мышью — и увидеть отчёт
- Либо нажать «Показать на примере», если журнала под рукой нет
- Понимаются журналы nginx, Apache, Caddy, Cloudflare

Почему это безопасно

Журнал никуда не отправляется. Это не обещание, а устройство: у страницы нет сервера, отправлять физически некуда. Всё считается на вашем компьютере.

Тот же расчёт доступен программой командной строки — числа совпадают, и это проверяется автоматически.

Кому это нужно

Клиент опознаётся по трём признакам, и нужны все три сразу:

1

Разные по тяжести запросы

В одном сервисе рядом живут мгновенные операции и тяжёлые: каталог и выгрузка, поиск и отчёт

2

Строгое требование к скорости

Обещание по времени отклика записано в договор, в требования к интерфейсу или в ожидания пользователей

3

Современная архитектура

Стек, где одно тяжёлое вычисление задерживает обработку всех остальных обращений

Где такие водятся

- Торговые площадки
- Платёжные шлюзы
- Финтех и банки
- Поиск и подбор
- Интерактивные панели
- Сервисы с выгрузками

Размер рынка в деньгах мы не считали и цифру называть не будем: любое число сейчас было бы выдумано. Как её получить — на слайде «Состояние».

Кто чувствует это острее всего

Три признака выше говорят, где живёт проблема. Но острее всех её чувствуют те, у кого меньше всего возможностей с ней разбираться.

Маленькие команды

Несколько человек, и все заняты продуктом. Выделенного инженера по инфраструктуре нет — и ещё долго не будет.

Новые продукты

Нагрузка только появилась и повела себя странно. Разбираться некогда: всё время уходит на то, ради чего стартап и затевался.

Те, кто не хочет вникать

И это нормально. Сегодня, чтобы понять, почему сервис иногда тормозит, надо стать специалистом по очередям и профилированию.

ЧТО МЫ ИМ ДАЁМ

Ответ, а не исследование. Перетащить журнал мышью — минута: без установки, без документации, без найма отдельного человека. Если проблемы нет — это тоже ответ, и он бесплатный для первых взаимодействий

Открытое ядро

Бесплатно, навсегда

Диагностика и маршрутизатор — открытый код. Это не щедрость, а расчёт: единственное наше преимущество в том, что нас можно проверить за десять минут, никого не спрашивая.

За деньги

То, что нужно, когда от инструмента уже зависят: обновление модели по свежему журналу, панель управления, оповещения о сбоях, интеграции и поддержка.

Принцип цены, который мы готовы записать в договор

Цена не превышает трети той экономии, которую диагностика показала на журнале самого клиента. Показала мало — значит, продавать нечего, и мы скажем это первыми.

*Конкретных тарифов пока не называем: платящих клиентов нет, и цена, придуманная до первой сделки, была бы выдумкой. **Первым, кто возьмёт продукт в работу, он достаётся бесплатно.***

Почему это не закрыто существующими средствами

Сразу скажем то, что в любом случае приходит на ум: сама идея предсказывать стоимость запроса не нова. Но существующие средства не решают задачу — и не потому, что они хуже, а потому что устроены иначе.

Кто	Чего не может
Обычные балансировщики	Реагируют на перегрузку, когда она уже случилась. Не знают, что именно этот запрос окажется тяжёлым
Промышленные шлюзы	Умеют учитывать стоимость, но требуют, чтобы её сообщал сам сервис, — то есть чтобы его переделали
Системы наблюдения	Показывают, что стало плохо, но не связывают это с параметром запроса и ничего не предпринимают
Автомасштабирование	Добавляет мощность, но не разбирает очередь: новый сервер поднимается десятки секунд

Наше место, сформулированное узко: модель стоимости выводится из обычного журнала, который пишется у всех, и не требует ни строчки изменений в сервисе клиента.

Проверено на четырёх уровнях

1

На чужом программном обеспечении

Базы данных и сервер, которые писали не мы. Инструмент сам, зная только времена ответа, нашёл, какая именно операция обходится дороже всего.

2

На данных Microsoft

1 801 682 426 вызовов настоящих клиентов облачной платформы. Это доказывает, что проблема массовая, а не выдумана под наш стенд.

3

В настоящем рабочем сервере

Числа про экономию сняты в той самой форме, в какой продукт приходит к клиенту, а не в упрощённой модели.

4

Любым, кто откроет репозиторий

Четыре проверки, каждая одной командой, десять минут. Ни одно число не попадает в наши тексты, пока не лежит в файле с данными.

И то, чем мы дорожим больше остального: первая версия проекта показывала результат вдвое лучше нынешнего. При проверке выяснилось, что он был завышен. Мы переделали всё с нуля — и честные числа получились скромнее заявленных.

Где это, вероятно, не поможет

Называем сами, потому что измерили. Продукт, у которого нет границ, обычно просто плохо проверен.

Если требование к скорости очень мягкое

Экономия по нашему же расчёту нулевая. Пакетной обработке и выгрузкам продукт не нужен, и мы не станем его туда продавать.

В некоторых случаях это размен, а не чистый выигрыш

Общая пропускная способность снижается примерно на 10-20%: тяжёлые обращения идут медленнее, зато лёгкие АБСОЛЮТНО перестают за ними стоять.

Если все запросы одинаковы

Раскладывать нечего, и наш подход проигрывает обычной схеме. Диагностика покажет это бесплатно и за минуту — до всякой покупки.

Боевых данных настоящей компании у нас нет

Это главный пробел. Ровно поэтому диагностика бесплатна и ничего не меняет в чужой системе.

Ни одно из этих ограничений не отменяет остального: там, где условия совпали, выигрыш измерен, велик и воспроизводится.

Что готово, чего нет и чего я прошу

Готово

- Диагностика работает и доступна всем
- Маршрутизатор замерен в рабочем сервере
- Все числа воспроизводятся

Ещё нет

- Ни одного платящего клиента
- Журналов настоящих компаний не видели
- Маршрутизатор — стабильно рабочий прототип, но не боевая версия

Команда

Один человек: студент МГТУ им. Н. Э. Баумана. Исследование, измерения, три формы продукта и эта презентация сделаны в одиночку

ЧЕГО ПРОШУ

Не денег на рост — рано. Прошу связей: возможности прогнать бесплатную диагностику на журналах трёх-пяти живых сервисов.

Журналы у команд уже есть, устанавливать ничего не надо, ответ получается за вечер. Это либо подтвердит, что рынок есть — и тогда его можно посчитать, — либо покажет, что проблема наша частная. Второе я скажу так же прямо.

СВЯЗАТЬСЯ

Покажите нам свой журнал

Если разброс у вас нашёлся — разберём вместе. Если не нашёлся — скажите тем более: отрицательный ответ для нас так же ценен.

Почта	ssd-tula@mail.ru
Telegram	@winlast
Продукт	winlast.github.io/loadlens
Исходный код	github.com/winlast/loadlens

ТЕХНИЧЕСКОЕ ПРИЛОЖЕНИЕ

Как это устроено и что именно измерено

Дальше — техническая часть. Для тех, кто захочет проверить механизм, а не только результат. Для понимания предыдущих слайдов она не нужна.

Почему вообще возникает задержка

Современные веб-движки обслуживают все соединения одним потоком выполнения. На ожидании внешних систем это прекрасно — потому отрасль на них и перешла. Но вычисление в таком потоке выполняется неделимо и на всё своё время останавливает обработку всех остальных запросов.

Один и тот же запрос — проверка пароля — задерживает соседние обращения по-разному, в зависимости от того, куда он попал:

9,3 мс

задержка соседям в сервере с потоками — там вычисление идёт параллельно с обработкой других запросов

124,7 мс

задержка соседям в однопоточном цикле — там они ждут чужое вычисление целиком

Собственное время выполнения при этом почти одинаковое: движки расходятся не в скорости, а в том, сколько вреда запрос причиняет соседям. Отсюда и решение: не пускать тяжёлое туда, где оно блокирует остальных.

Откуда берётся число, которого ещё нет

Стоимость надо узнать до обработки запроса. Единственный источник, который есть у всех и ничего не требует, — журнал прошлых обращений.

1

Читаем журнал

Адрес, параметры запроса и время ответа. Больше ничего не нужно: ни доступа к коду, ни встроенных замеров.

2

Подбираем зависимость

Метод наименьших квадратов в логарифмах — многомерная линейная регрессия. Около шестидесяти строк без единой библиотеки.

3

Проверяем на отложенных данных

Модель настраивается на одной половине журнала и проверяется на другой, которую не видела.

Насколько это работает

На чужой базе оценка по адресу ошибается на 177,1 %, а оценка по признакам запроса — на 33,1 %. Инструмент при этом сам, без подсказки, определил, какая операция дороже всего, — и это оказалась не та, о которой думали.

*Нейронной сети в продукте нет. Она есть в исследовательской части, но в продукт не пошла: **решение** ею принимается в сто раз медленнее и само становится узким местом.*

Числа, на которых держится экономия

Один и тот же рабочий сервер, тот же трафик, те же обработчики. Отличается только способ выбора, куда отправить запрос.

	обычная балансировка	LoadLens
быстрые запросы, медиана	103,9 мс	11,3 мс
медленные запросы, порог 95 %	852,4 мс	73,0 мс
доля медленнее 200 мс	36,9 %	0,0 %
запросов в секунду при обещании 100 мс	37,8	59,2
общая пропускная способность	67,7	53,4

Последняя строка — цена, которую мы платим. Общая пропускная способность немного падает: это осознанный размен, а не побочный эффект. Мы показываем её в той же таблице, а не прячем.

Пустая ячейка, которую мы можем занять

Промышленные шлюзы уже научились брать плату по стоимости запроса: квота списывается не по числу обращений, а по их тяжести. Механизм готов и работает.

Но откуда берётся само число — не решено ни у кого. В документации прямо сказано: автоматической оценки стоимости нет, нужно либо вписать значение руками, либо чтобы его вычислил и передал посторонний сервис.

Этим посторонним сервисом и должен стать LoadLens. Связка с промышленным шлюзом пока не построена — это ближайший шаг.

- Строить свой ограничитель не нужно — он уже есть и лучше нашего
- Пустует ровно та ячейка, которую можем заполнить мы: кто скажет, сколько стоит вот этот запрос, до того как он обработан
- Подключение идёт через стандартную точку расширения, а не через нашу самоделку

Что измерено, а что пока нет

Измерено и воспроизводится

- Механизм задержки и её разница между движками
- Разброс стоимости внутри одного адреса — на чужом ПО
- Предсказуемость стоимости по признакам запроса
- Выигрыш в настоящем рабочем сервере
- Цена размена: минус пятая часть пропускной способности
- Массовость проблемы — на данных Microsoft

Не измерено

- Боевой трафик настоящей компании
- Поведение при росте числа серверов — предполагается, не измерено
- Готовность рынка платить и его размер
- Предсказание стоимости на данных Microsoft: параметров запросов там нет, проверена только сама проблема

Правило проекта: ни одно число не попадает в текст, пока не лежит в файле с данными. Соответствие проверяется автоматически при каждой сборке — включая числа этой презентации.